



Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение
высшего образования
«Московский государственный технический университет имени
Н.Э. Баумана
(национальный исследовательский университет)»
(МГТУ им. Н.Э. Баумана)

ФАКУЛЬТЕТ «ИНФОРМАТИКА И СИСТЕМЫ УПРАВЛЕНИЯ»

КАФЕДРА «ПРОГРАММНОЕ ОБЕСПЕЧЕНИЕ ЭВМ И ИНФОРМАЦИОННЫЕ ТЕХНОЛОГИИ»

Отчёт по лабораторной работе №2 по дисциплине «Анализ алгоритмов»

Тема Сравнительный анализ рекурсивного и нерекурсивного алгоритмов

Студент Попов Ю.А.

Группа ИУ7-52Б

Преподаватели Волкова Л.Л., Строганов Ю.В.

Москва, 2025

СОДЕРЖАНИЕ

ВВЕДЕНИЕ	3
1 Аналитическая часть	4
1.1 Постановка задачи	4
1.2 Рекурсия	4
1.3	4
2 Конструкторская часть	5
2.1 Описание алгоритмов	5
2.2 Оценка трудоёмкости алгоритмов	7
2.3 Трудоёмкость стандартного алгоритма	7
2.4 Трудоёмкость алгоритма Винограда	8
2.5 Трудоёмкость оптимизированного алгоритма Винограда	8
2.6 Используемые типы данных	9
2.7 Оценка памяти	10
2.8 Структура программы	10
3 Технологическая часть	11
3.1 Требование к программному обеспечению	11
3.2 Средства реализации	11
3.3 Реализация алгоритмов	11
3.4 Тестирование ПО	13
4 Исследовательская часть	14
4.1 Технические характеристики	14
4.2 Реализация замеров	14
4.3 Оценка времени работы алгоритмов	14
ЗАКЛЮЧЕНИЕ	16
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ	17

ВВЕДЕНИЕ

Целью данной работы является выполнить сравнительный анализ рекурсивного и нерекурсивного алгоритмов.

Для достижения цели поставлены следующие задачи:

- разработать рекурсивный и нерекурсивный алгоритмы решения задачи нахождения максимального числа последовательности;
- описать средства разработки и инструменты замера процессорного времени выполнения реализации алгоритмов;
- реализовать разработанные алгоритмы;
- выполнить тестирование реализации алгоритмов;
- выполнить теоретическую оценку затрачиваемой реализацией каждого алгоритма памяти (для рекурсивного алгоритма — на материале анализа высоты дерева рекурсивных вызовов и оценки затрачиваемой на один вызов функции памяти);
- выполнить замеры процессорного времени выполнения реализации алгоритмов в зависимости от варьируемого входа;
- оценить трудоёмкость двух алгоритмов или их реализаций в худшем случае;
- сравнить результаты замеров процессорного времени и оценки трудоемкости;
- сделать выводы из сравнительного анализа реализации рекурсивного и нерекурсивного алгоритмов решения одной и той же задачи, заданной вариантом, по критериям ёмкостной эффективности (на материале теоретической оценки пиковой временной эффективности).

1 Аналитическая часть

1.1 Постановка задачи

В лабораторной работе №2 необходимо провести сравнительный анализ рекурсивного и нерекурсивного алгоритма. По варианту необходимо найти максимальное число в последовательности чисел.

1.2 Рекурсия

Рекурсия в программировании представляет собой технику, при которой функция вызывает саму себя для решения подзадачи того же типа. Такой подход особенно удобен при работе с последовательностями или древовидными структурами, где задача может быть естественным образом разбита на более мелкие экземпляры [1]. **Рекурсивный алгоритм** — алгоритм, определяемый через себя [2].

1.3 Механизм выполнения рекурсивных вызовов

Каждый

2 Конструкторская часть

В данной части будут реализованы блок-схемы алгоритмов поиска максимального числа в последовательности, произведена оценка их трудоёмкости и затрачиваемая памяти.

2.1 Описание алгоритмов

На основе аналитической части были спроектированы алгоритмы умножения матриц. Блок-схемы этих алгоритмов приведены на рисунке 2.1 (стандартный), рисунке 2.2 (Винограда) и рисунке 2.3 (Оптимизированного Винограда).

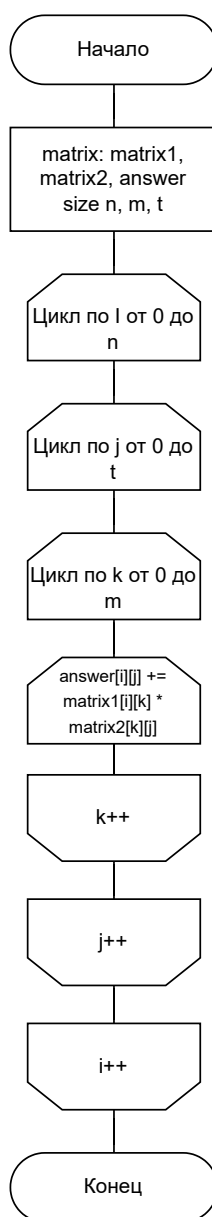


Рисунок 2.1 — Схема стандартного алгоритма умножения матриц

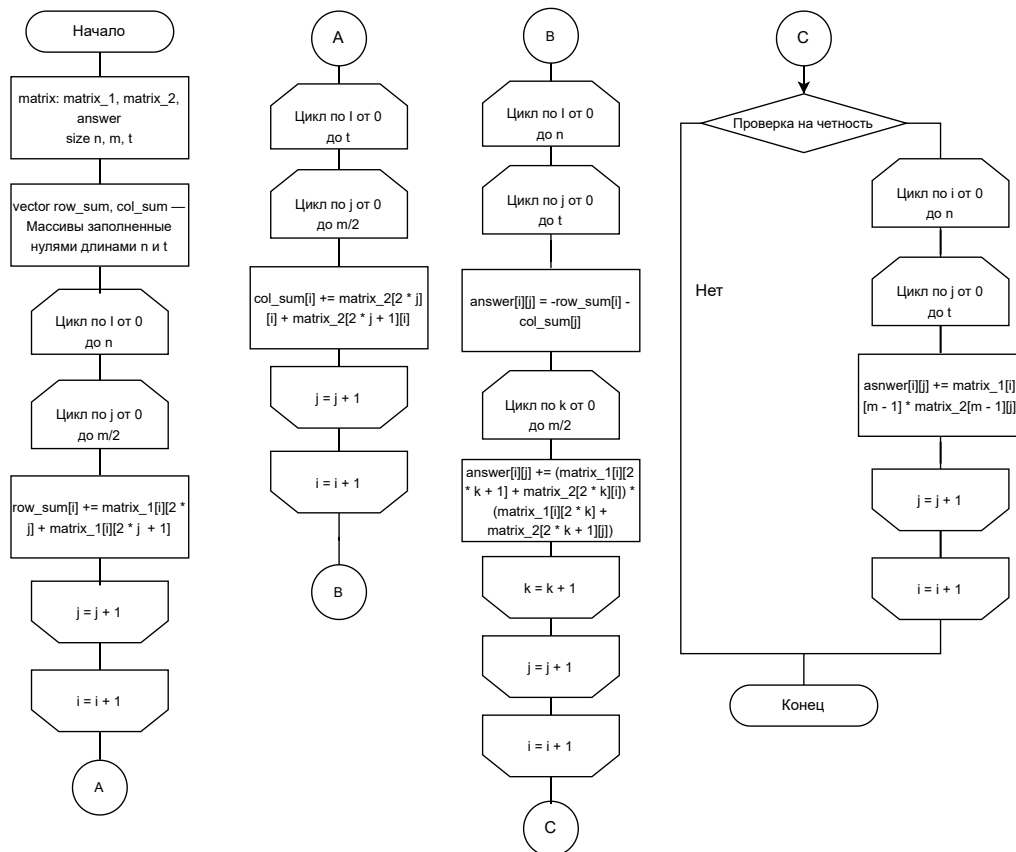


Рисунок 2.2 — Схема алгоритма Винограда

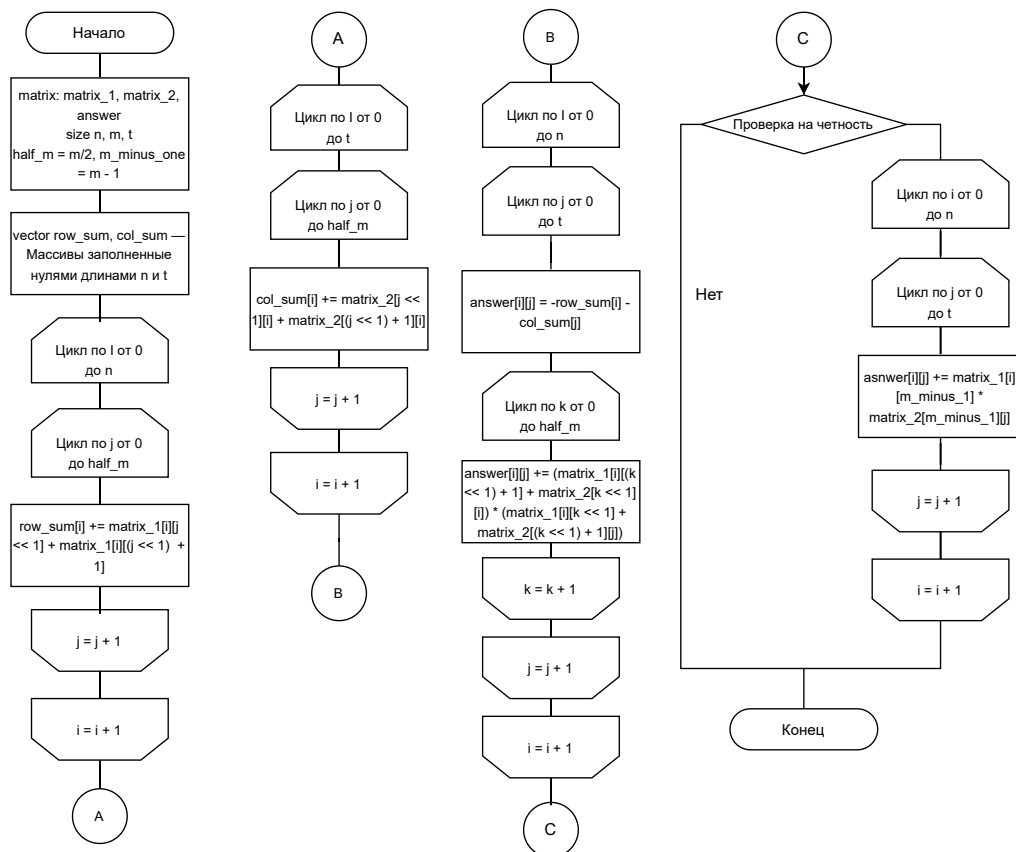


Рисунок 2.3 — Схема алгоритма Винограда

2.2 Оценка трудоёмкости алгоритмов

Введена модель для оценки трудоёмкости

1) Трудоёмкость базовых операций

— Определим трудоёмкость следующих операций равной 1:

$+$, $-$, $!$, $! =$, $==$, $\neq$, $\leq$, $\geq$, $>$, $<$, $+$, $=$, $-$, $=$, $[]$, $||$, $\&\&$, $<<$

— Определим трудоёмкость следующих операция равной 2:

$*$, $/$, $*$, $=$, $/$, $=$, $\%$

2) Трудоёмкость циклов

$$f = f_{\text{иниц.}} + f_{\text{сравн.}} + N_{\text{итер}} \cdot (f_{\text{тела}} + f_{\text{инкремента}} + f_{\text{сравн.}}) \quad (2.1)$$

3) Трудоёмкость условного оператора, приняв трудоёмкость условного перехода за ноль

$$f_{\text{if}} = f_{\text{условия}} + \begin{cases} \min(f_{\text{true}}, f_{\text{false}}), & \text{в лучшем случае} \\ \max(f_{\text{true}}, f_{\text{false}}), & \text{в худшем случае} \end{cases} \quad (2.2)$$

2.3 Трудоёмкость стандартного алгоритма

Трудоёмкость стандартного алгоритма состоит из :

— трудоёмкость цикла по i от 1 до n :

$$f = 2 + n * (2 + f_{\text{body}}) \quad (2.3)$$

— трудоёмкость цикла по j от 1 до t :

$$f = 2 + t * (2 + f_{\text{body}}) \quad (2.4)$$

— трудоёмкость цикла по k от 1 до m :

$$f = 2 + k * (2 + f_{\text{body}}) \quad (2.5)$$

— трудоёмкость тела цикла по k :

$$f = 2 + 1 + 2 + 2 + 2 = 11 \quad (2.6)$$

Таким образом, общая трудоёмкость равна

$$f = 2 + n * (2 + 2 + t * (2 + 2 + 11k)) \quad (2.7)$$

$$f = 2 + n * (4 + t * (4 + 11k)) = 2 + n * (4 + 4t + 11tk) \quad (2.8)$$

$$f = 11ntk + 4nt + 4n + 2 \approx ntk \approx O(N^3) \quad (2.9)$$

2.4 Трудоёмкость алгоритма Винограда

Трудоёмкость стандартного алгоритма состоит из :

— выделение памяти и инициализация векторов:

$$f = 17nm/2 + 17tm/2 + 6t + 6n + 4 \quad (2.10)$$

— трудоёмкость цикла по i от 1 до n :

$$f = 2 + n * (2 + f_{body}) \quad (2.11)$$

— трудоёмкость цикла по j от 1 до t :

$$f = 2 + t * (9 + f_{body}) \quad (2.12)$$

— трудоёмкость цикла по k от 1 до $m/2$:

$$f = 2 + 27 * m/2 \quad (2.13)$$

— трудоёмкость проверки на нечетность

$$f_{if} = 3 + \begin{cases} 0, & \text{в лучшем случае} \\ 13nt + 4n + 2, & \text{в худшем случае} \end{cases} \quad (2.14)$$

Таким образом, общая трудоёмкость равна:

$$f = \begin{cases} 27nmt/2 + 17nm/2 + 17tm/2 + 11nt + 6t + 10n + 6, & \text{в лучшем случае} \\ 27nmt/2 + 17nm/2 + 17tm/2 + 24nt + 6t + 14n + 8 & \text{в худшем случае} \end{cases} \quad (2.15)$$

$$f = 13.5nmt + 8.5nm + 8.5mt + 24nt \approx O(N^3) \quad (2.16)$$

2.5 Трудоёмкость оптимизированного алгоритма Винограда

Трудоёмкость стандартного алгоритма состоит из :

— инициализация переменных:

$$f = 3$$

— выделение памяти и инициализация векторов:

$$f = 13nm/2 + 13tm/2 + 4n + 4t + 4 \quad (2.17)$$

— трудоёмкость цикла по i от 1 до n :

$$f = 2 + n * (2 + f_{body}) \quad (2.18)$$

— трудоёмкость цикла по j от 1 до t :

$$f = 2 + t * (9 + f_{body}) \quad (2.19)$$

— трудоёмкость цикла по k от 1 до $m/2$:

$$f = 2 + m/2 * (2 + f_{body}) \quad (2.20)$$

— трудоёмкость тела цикла

$$f = 2 + 1 + 2 + 1 + 1 + 1 + 2 + 1 + 2 + 2 + 1 + 1 + 2 + 1 + 1 = 21 \quad (2.21)$$

— трудоёмкость проверки на нечетность

$$f_{if} = 3 + \begin{cases} 0, & \text{в лучшем случае} \\ 11nt + 4n + 5, & \text{в худшем случае} \end{cases} \quad (2.22)$$

Таким образом, общая трудоёмкость равна

$$f = 13nm/2 + 13tm/2 + 4n + 4t + 6 + n * (4 + t * (11 + m/2 * 23)) \quad (2.23)$$

$$f = 13nm/2 + 13tm/2 + 4n + 4t + 6 + n * (4 + 11t + 23tm/2) \quad (2.24)$$

$$f = 13nm/2 + 13tm/2 + 4n + 4t + 6 + 4n + 11tn + 23tmn/2 \quad (2.25)$$

$$f = 23tmn/2 + 13nm/2 + 13tm/2 + 11tn + 4t + 8n + 6 \quad (2.26)$$

$$f = \begin{cases} 23tmn/2 + 13nm/2 + 13tm/2 + 11tn + 4t + 8n + 9, & \text{в лучшем случае} \\ 23tmn/2 + 13nm/2 + 13tm/2 + 22nt + 4t + 12n + 11 & \text{в худшем случае} \end{cases} \quad (2.27)$$

$$f = 11.5nmt + 6.5nm + 6.5mt + 11nt \approx O(N^3) \quad (2.28)$$

2.6 Используемые типы данных

При реализации алгоритмов использовались следующие типы данных:

- размеры матрицы: `size_t`;
- тип данных матрицы: `int`;

— тип данных вспомогательных векторов: `int`;

2.7 Оценка памяти

Пусть `matrix` — матрица целых чисел

— размерность входных матриц $n * m$ и $m * t$;

— размерность результирующей матрицы $n * t$;

Для стандартного алгоритма умножения будет затрачена память на:

— размеры матрицы — $3 * sizeof(size_t)$;

— матрицы — $(n * m + m * t + n * t) * sizeof(int)$;

Для алгоритма Винограда создаются два дополнительных массива, для которых используется $(n + t) * sizeof(int)$ памяти. Для оптимизации алгоритма используется две вспомогательные переменные размером $sizeof(size_t)$.

В стандартном алгоритме не используется дополнительная памяти на массивы, что даёт ему выигрыш по сравнению с алгоритмом Виноградова, хоть и незначительный.

2.8 Структура программы

Программное обеспечение будет состоять из нескольких частей:

— главный модуль — предоставляет возможность выбора режима работы программы, а также обеспечивает запуск программы;

— интерфейс — модуль для взаимодействия пользователя с программой. Пользователь может выбрать режим работы, а также ввести входные данные для умножения матриц или тестовых замеров;

— модуль, содержащий функции для работы с матрицами — предоставляет возможность объявить, проинициализировать, прочесть, заполнить случайными числами и совершить арифметические операции над типом данных;

— модуль для совершения замеров процессорного времени выполнения умножения матриц;

Вывод

На основе аналитической части были реализованы блок-схемы алгоритмов умножения матриц, произведена оценка их трудоёмкости, вычислена затрачиваемая памяти, под каждый анализ, а также продуманы модули программ.

3 Технологическая часть

В данном разделе будут описаны требования к реализации программного обеспечения, выбору языков программирования и тестированию.

3.1 Требование к программному обеспечению

Программное обеспечение должно удовлетворять следующим критериям:

- программа должна обеспечить возможность ввести две матрицы для их умножения. Для успешного ввода пользователь должен иметь возможность задать 2 размера матрицы и поэлементно ввести содержимое матрицы;
- для замеров времени выполнения, ПО должно обеспечить возможность ввода начального и конечного размера матриц, а также ввести шаг замеров;
- программа должна иметь возможность умножения двух матриц тремя алгоритмами: стандартным, Винограда и оптимизированным по варианту алгоритмом Винограда.

3.2 Средства реализации

В качестве средства реализации был выбран язык C++. Для разработки была выбрана среда VS Code. Для сборки проектов была выбрана утилита CMake. Для корректной работы с системой pipeline был написан Makefile. Средством построения графиком и сборки результатов тестов выбран язык Python, в виду распространённости библиотек для работы с данными.

3.3 Реализация алгоритмов

Ниже приведены листинги реализации алгоритмов умножения матриц. На листинге 3.1 (стандартный), 3.2 (Винограда), 3.3 (Оптимизированный Винограда)

```
for (size_t i = 0; i < n; i++) {
    for (size_t j = 0; j < t; j++) {
        for (size_t k = 0; k < m; k++) {
            answer._matrix[i][j] += this->_matrix[i][k] * other._matrix[k][j];
        }
    }
}
```

Листинг 3.1 — Стандартный алгоритм умножения матриц

```

    for (size_type i = 0; i < n; i++) {
    for (size_type j = 0; j < m / 2; j++) {
        row_sum[i] += (this->_matrix[i][2 * j] * this->_matrix[i][2 * j +
            1]);}}
for (size_type i = 0; i < t; i++) {
    for (size_type j = 0; j < m / 2; j++) {
        col_sum[i] += (other._matrix[2 * j][i] * other._matrix[2 * j + 1][i
            ]));}}

for (size_t i = 0; i < n; i++) {
    for (size_t j = 0; j < t; j++) {
        answer._matrix[i][j] = -row_sum[i] - col_sum[j];
        for (size_t k = 0; k < m / 2; k++) {
            answer._matrix[i][j] +=
                (this->_matrix[i][2 * k + 1] + other._matrix[2 * k][j]) *
                (this->_matrix[i][2 * k] + other._matrix[2 * k + 1][j]));}}}
...

```

Листинг 3.2 — Стандартный алгоритм умножения матриц

```

for (size_type i = 0; i < n; i++) {
    for (size_type j = 0; j < half_m; j++) {
        row_sum[i] += (this->_matrix[i][j << 1] * this->_matrix[i][(j << 1)
            + 1]); }}
for (size_type i = 0; i < t; i++) {
    for (size_type j = 0; j < half_m; j++) {
        col_sum[i] += (other._matrix[j << 1][i] * other._matrix[(j << 1) +
            1][i]); }}

for (size_t i = 0; i < n; i++) {
    for (size_t j = 0; j < t; j++) {
        answer._matrix[i][j] = -row_sum[i] - col_sum[j];
        for (size_t k = 0; k < half_m; k++) {
            answer._matrix[i][j] +=
                (this->_matrix[i][(k << 1) + 1] + other._matrix[k << 1][j]) *
                (this->_matrix[i][k << 1] + other._matrix[(k << 1) + 1][j]));}}}
...

```

Листинг 3.3 — Оптимизированный алгоритм Винограда

3.4 Тестирование ПО

Для тестирования ПО было выбрано 4 тестовых случая для каждого алгоритма. Тестовые случаи указанные в таблице 3.1

Таблица 3.1 — Примеры умножения матриц различной размерности

Матрица А	Матрица В	Результат (матрица)	Код
(2)	(3)	(6)	OK
$\begin{pmatrix} 1 & 2 \\ 3 & 4 \end{pmatrix}$	$\begin{pmatrix} 5 & 6 \\ 7 & 8 \end{pmatrix}$	$\begin{pmatrix} 19 & 22 \\ 43 & 50 \end{pmatrix}$	OK
$\begin{pmatrix} 1 & 2 \\ 3 & 4 \\ 5 & 6 \end{pmatrix}$	$\begin{pmatrix} 7 & 8 & 9 \\ 10 & 11 & 12 \end{pmatrix}$	$\begin{pmatrix} 27 & 30 & 33 \\ 61 & 68 & 75 \\ 95 & 106 & 117 \end{pmatrix}$	OK
$\begin{pmatrix} 1 & 2 \\ 3 & 4 \end{pmatrix}$	(1 2)	—	ERROR

Все тесты прошли успешно.

Вывод

На основе конструкторской части были реализованы и протестированы алгоритмы умножения матриц.

4 Исследовательская часть

В данном разделе будет проведено исследование процессорного времени выполнения разработанных алгоритмов.

4.1 Технические характеристики

Исследования проводились на устройстве со следующими характеристиками[3]

- операционная система: MacOS Sequoia 15.6.1 (24G90);
- процессор: Apple M3, 8 ядер;
- оперативная память: 16 Гигабайт, LPDDR5;

4.2 Реализация замеров

Во время выполнения замеров устройство было подключено к источнику питания и не выполняло сторонних процессов. Замеры проводились с помощью встроенной библиотеки *chrono*. Для каждого размера матрицы каждого алгоритма запуск замера проводился 100 раз, после чего время выполнения усреднялось. Для проведения замеров была отключена оптимизация компилятора флагом `-O0`[4].

```
auto start = std::chrono::high_resolution_clock::now();
for (size_t j = 0; j < EXP_COUNT; j++) {
    matrix_1.multiply(matrix_2, answer);
}
auto end = std::chrono::high_resolution_clock::now();
auto duration = std::chrono::duration_cast<std::chrono::microseconds>(
    end-start);
```

Листинг 4.1 — Функция замера времени выполнения

4.3 Оценка времени работы алгоритмов

В таблице 4.1 представлены замеры процессорного времени работы алгоритмов на квадратных матрицах размером от 15 до 450 с шагом 15. Шаг выбран нечётным для того, чтобы в выборку попали как наилучшие, так и наихудшие случаи.

На рисунке 4.1 продемонстрирован график зависимости процессорного времени умножения от размера матрицы на случайных данных. В результате замеров было выяснено, что оптимизированный алгоритм Винограда работает быстрее на 15% чем классический алгоритм Винограда, а также на 40% чем стандартный.

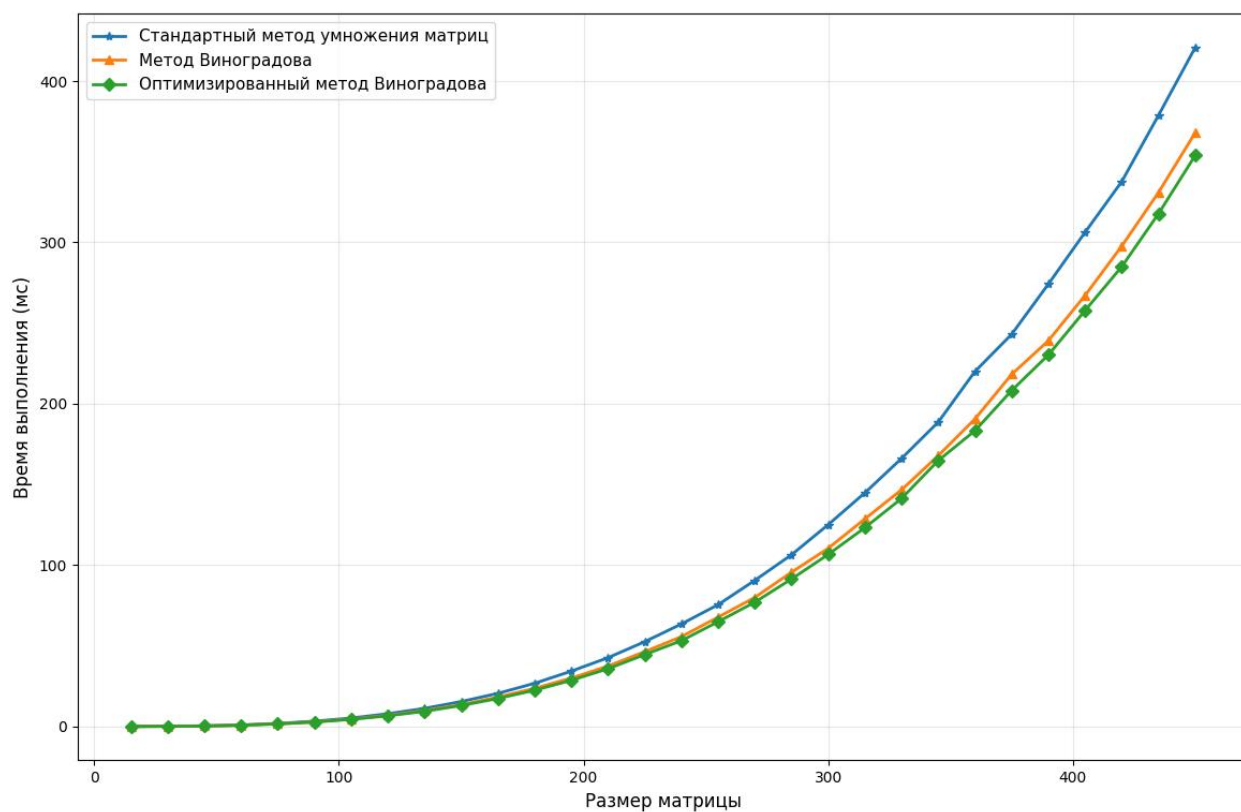


Рисунок 4.1 — Зависимость времени выполнения от размеров матриц

Вывод

При реализации исследовательской части было выяснено, что оптимизированный алгоритм Винограда выполняется быстрее:

- на 5% в худшем случае и 17% в лучшем случае чем классический алгоритм Винограда;
- на 15% в худшем случае и 45% в лучшем случае чем стандартный алгоритм;

ЗАКЛЮЧЕНИЕ

В ходе выполнения лабораторной работы были выполнены следующие задачи:

- выбраны алгоритмы умножения матриц;
- оптимизирован алгоритм Винограда методом по варианту;
- введена модель вычислений и выполнена оценка трудоёмкости;
- реализованы алгоритмы на выбранном языке программирования;
- проведён сравнительный анализ алгоритмов;

В результате исследования получено, что стандартный алгоритм является самым медленным из рассмотренных. Несмотря на то, что все алгоритмы имеют сложность $O(N^3)$, их трудоёмкость различается. Таким образом оптимизированный алгоритм Виноградова в среднем выполняется быстрее на 25%, чем стандартный, при небольшом увеличении затрачиваемой оперативной памяти на 2 массива.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Thomas H Cormen, Charles E Leiserson, Ronald L Rivest, and Clifford Stein. Introduction to algorithms. MIT press, Cambridge, Massachusetts; London, 2009. — 1313 с.
2. *Рекурсия* [Электронный ресурс]. URL: <https://zns.susu.ru/IT/ВР/10.pdf>(дата обращения: 26.10.2025).